

Funzioni in Python

Supponiamo di calcolare su Vs Code tutti i numeri interi tra 1 a 10 , da 20 a 37 , da 35 a 49 .

Expected Output:

- Somma gli interi da 1 a 10 = 55
- Somma gli interi da 20 a 37 = 513
- Somma gli interi da 35 a 49 = 630

Per fare ciò normalmente bisognerebbe fare questo:



Python

```
1  somma:int = 0
2
3  for number in range (1,11):
4
5      somma += number
6
7      print(somma)
8
9
10
11 sum:int= 0
12
13 for numbers in range (20, 38):
14
15     sum = sum + numbers
16
17     print(sum)
18
19
20
21
22 summa:int = 0
23
24 for i in range (35, 50):
25
26     summa+=i
27
28     print(summa)
```

Da notare come ho dovuto riscrivere per tre volte lo stesso codice, l'unica cosa che cambia è il range dei numeri, cioè i parametri che la funzione `.range()` assume.

Come abbiamo visto nell'esempio, alle volte ci troviamo con la necessità di riscrivere più e più volte lo stesso codice, quindi dobbiamo trovare uno strumento che ci permette di scrivere il codice lo stesso codice e richiamarlo più volte, Come per i cicli che iterano ad esempio più volte il comando al suo interno.

Le funzioni

Permettono di eseguire un insieme di istruzioni più volte senza doverlo riscrivere.

Grazie alle funzioni, possiamo definire un blocco di codice una sola volta e richiamarlo ogni volta che serve, rendendo il programma più organizzato e leggibile.

Cos'è una funzione?

Una funzione è un insieme di istruzioni progettato per eseguire una specifica operazione.

A questo gruppo di istruzioni viene assegnato un **nome**, che ci permette di richiamarle all'interno del codice senza doverle riscrivere ogni volta.

✓ Vantaggi delle funzioni

- **Modularità :**
Suddividono il codice in parti più piccole e organizzate.
- **Riutilizzabilità :**
Permettono di evitare la duplicazione del codice.
- **Flessibilità :**
Possono ricevere **valori in input**, eseguire operazioni e **restituire un valore in output**.

Sintassi

La sintassi di una funzione di Python è la seguente:

1. `def :`

Keyword che indica l'inizio della definizione di una funzione.

Quindi "informa" Python che si sta definendo una funzione.

2. `functionName` :

Il nome assegnato alla funzione.

3. `(list of parameters)` :

Elenco opzionale di parametri che la funzione può ricevere in input.

4. `::`

I due punti indicano l'inizio del blocco di codice della funzione.

5. **Corpo della funzione (Blocco di codice indentato):**

Contiene le istruzioni che verranno eseguite quando la funzione viene chiamata.



Python

```
1 def functionName (list of parameters):  
2     instructions for function body
```

Se una variabile e una funzione hanno lo stesso nome, possiamo distinguerle grazie alle **parentesi tonde**:

- Se il nome è seguito da `()` , si tratta di una funzione.
- Se il nome è usato senza parentesi, è una variabile.

Alcune funzioni non richiedono particolari parametri input.

Ad esempio le funzioni della classe stringa(`.lower()` , `.upper()` , etc), questo perché sono un particolare tipo di funzioni (dette built-in) che non richiedono parametri in input poiché agiscono sull tipo di dato(in questo caso la stringa) su cui sono chiamate.

Ovviamente non tutte le funzioni native di Python non richiedono parametri in input , ad esempio la funzione `.range()` accetta tre valori in input.

🚫 Nome delle variabili e nomi delle funzioni >

Come abbiamo già detto Python è un linguaggio orientato agli oggetti, per cui i nomi delle variabili non sono dei recipienti ma dei puntatori.

Partendo da questi due concetti se si dichiara una variabile e le si assegna un valore:



Python

```
1 #Dichiaro una variabile  
2 number_value:int = 5  
3 print(number_value) #Output:5  
4 print(type(number_value)) #Output:class int
```

```

5
6  #Dichiaro una funzione con lo stesso >nome
7  def number_value():
8      print("Hello")
9
10 print(number_value) #Output: function number_value at
    0x00000021A9CCACA40
11 print(type(number_value)) #Output: class function

```

Quindi come possiamo vedere dopo aver ridefinito `number_value` come funzione, il nome non fa più riferimento all'intero `5`, ma all'oggetto funzione

Esempio pratico di una funzione in Python

Prendiamo come esempio questa funzione:



Python

```

1  def sum_range (a:int, b:int):
2      result:int = 0
3      for i in range (a, b+1):
4          result = result +i
5      return result

```

In questo caso voglio generalizzare la somma di tutti i numeri interi all'interno di un certo intervallo estremi inclusi, quindi ho bisogno di due valori `a` e `b`:

1. Parametri in input:

- `a` (intero) → estremo inferiore dell'intervallo.
- `b` (intero) → estremo superiore dell'intervallo.

2. Inizializzazione della variabile `result` a 0.

3. Ciclo `for`:

- Scorre i numeri da `a` a `b` (incluso `b`, quindi si usa `range(a, b+1)`).
- Ad ogni iterazione, aggiunge il valore corrente alla variabile `result`.

4. Restituzione del risultato con `return result`.

Come per la variabile che le si assegna un nome appropriato per rappresentare al meglio possibile il valore di quel dato, stessa cosa lo si fa per le funzioni.

Per riutilizzare le funzioni devo (in Python si dice richiamare o invocare una funzione) una funzione devo scrivere il nome della funzione dopo averla definita:



Python

```
1 def sum(a:int, b:int):
2     result:int = 0
3     for i in range(a, b+1):
4         result = result + i
5     return result
6
7 mysum= sum (1,10)
```

Richiamo la funzione e la salvo nella variabile `mysum` per salvarla in memoria, come argomenti messi in input vi sono i numeri `1` e `10` che corrispondono rispettivamente ai parametri `a` e `b` della funzione.

Se vado a sostituire `1` e `10` all'interno della mia funzione?

Cambio i parametri per calcolare la somma tra i numeri interi.

Una volta definita la definizione posso riutilizzarla quante volte mi pare.

Vediamo come possiamo riscrivere meglio questo pezzo di codice:



Python

```
1 def sum(a:int, b:int):
2     # define a sum to be computed
3     result:int = 0
4     # compute a sum
5     for i in range(a,b+1):
6         result = result + i
7     # return the value of sum as the output of the sum function
8     return result
9     # use sum function to compute a sum of integers from 1 to 10
10    print(f"Sum from 1 to 10 is {sum(1,10)}")
11    # use sum function to compute a sum of integers from 20 to 37
12    print(f"Sum from 20 to 37 is {sum(20,37)}")
13    # use sum function to compute a sum of integers from 35 to 49
14    print(f"Sum from 35 to 49 is {sum(35,49)}")
```

In questo caso siccome la funzione `sum` ci restituisce un valore la posso anche scrivere direttamente all'interno di un `print()` formattato.

Posso anche annidare funzioni tra loro

Esercizio

Definiamo una sottrazione `subtract` :

- Deve prendere 2 parametri in input
- dentro la funzione deve sottrarre i due parametri
- in seguito ritorna la funziona



Python

```
1 def subtract (a:int, b:int):
2
3     result:int= 0
4
5     result= a-b
6
7     return result
8
9 print(f"La sottrazione tra 10 e 5 è: {subtract(10,5)}")
```

User defined functions e le built-in functions

Python ha diversi tipi di funzioni

- **built-in functions:**

Funzioni integrate in Python (`print()` ,etc.). ^built-inFun

Sono le più efficienti in quanto offerte già da Python quando lo si installa.

- **User defined functions:**

Sono le funzioni scritte dal programmatore.

Consentono ai programmatori di scrivere blocchi di codice riutilizzabili per rendere il codice più pulito e più leggibile, quindi possiamo dividere un problema complesso in parti più piccole e semplici da gestire.

Queste funzioni possono essere integrate in librerie esterne come `pyp` o `pip` , ad esempio.

🚫 Nomi delle funzioni

1. Non dare nomi alle funzioni User defined delle funzioni built-in (come ad esempio `min` , `max` , `range()` , etc.)

2. Non usare un nome assegnato a una variabile precedentemente e poi dare lo stesso nome alla funzione

Hint

È consigliabile non fare funzioni troppo generabili che funzionano per ogni dato, le funzioni devono avere un focus specifico per quell'operazione su quei dati specifici, esattamente come per le liste, nelle quali posso inserire ogni tipo di dato(integer, float, stringa, booleano), ma è sconsigliabile farlo. Come per le esercizio della funzione di sottrazione; è meglio creare una funzione generale che dati due interi me li sottrae

Il Return

Ogni volta che una funzione contiene un'istruzione `return`, **essa restituisce un valore in output.**

Perché è importante restituire un valore?

Se una funzione non ritorna un valore, non può essere sfruttata al meglio. Invece, se la funzione restituisce un valore, possiamo:

- **Salvare il risultato in una variabile,**
- **Stampare il risultato in output,**
- **Usarlo in altre operazioni.**

In altre parole, una funzione che non ha la parola chiave `return` non fornirà alcun risultato che possiamo utilizzare al di fuori di essa.

Questo significa che quando definisco la funzione `subtract`, questa mi ritorna il valore della sottrazione `a-b`.

Hint

In alto a destra c'è una freccia che è scritta come `->:int` questo significa che la funzione restituisce un intero

```
def subtract(a:int, b:int) -> int:
    result:int = a - b
    return result
```

function Python.png

Questo tipo di nomenclatura posso scriverla in Python.

Return vs Print

Immaginiamo di riprendere la funzione `subtract()` e di passarle in input `10` e `5`, sapendo che la sottrazione dà `5`.

- Con `print`:
il valore della sottrazione viene stampato in output, ma non è accessibile altrove nel programma.
- Con `return`:
Invece, la funzione restituisce il valore della sottrazione, permettendoti di salvarlo, utilizzarlo in altre operazioni o stamparlo in seguito.

Se non salvi il risultato della funzione che utilizza `return` in una variabile, la sottrazione viene comunque eseguita, ma non puoi usarla di nuovo a meno che non la salvi prima.

Prendiamo ad esempio una funzione senza il `return`:



Python

```
1 def greet (name:str) →None:
2     print(f"Hello {name}")
3 greet ("Angela")
4 >>>Hello Angela
5 Print(type(greet(Angela)))
6 >>> <class 'NoneType'>
```

In questo esempio:

- La funzione `greet` stampa un messaggio di saluto, ma non restituisce un valore (il ritorno implicito è `None`).
- Se proviamo a fare `print(type(greet("Angela")))`, il tipo restituito è `NoneType`, perché la funzione non ritorna nulla, ma solo stampa il messaggio.

In altri linguaggi di programmazione bisogna specificare i valori che ritorna quella funzione.

Per fare un ulteriore esempio:



Python

```
1 def greet(name):
2
3     print(f"Hello, {name}")
4
5 user_name = greet("Angela")
6 print(user_name)
```

- La funzione `greet("Angela")` riceve l'argomento `"Angela"` e **stampa** `Hello, Angela` grazie al comando `print` all'interno della funzione.
- Tuttavia, la funzione `greet()` non ha un `return` esplicito, quindi **non restituisce nulla**. Per impostazione predefinita, se una funzione non ha un `return`, Python restituisce `None`.
- Quando provi a **stampare** `user_name`, il valore assegnato a `user_name` è `None`, perché la funzione non restituisce alcun valore, quindi `user_name` contiene `None`.

Mentre se io scrivessi:



Python

```
1 def greet(name):
2     return(f"Hello, {name}")
3
4 user_name = greet("Angela")
5 print(user_name)
```

In questo caso:

- La funzione `greet("Angela")` riceve l'argomento `"Angela"`.
- La funzione **resta nella memoria**, il risultato tramite `return f"Hello, {name}"`, che restituisce il messaggio `"Hello, Angela"`.
- La variabile `user_name` riceve questo valore di ritorno e quindi contiene la stringa `"Hello, Angela"`.
- Quando poi stampi `user_name`, il valore che contiene viene visualizzato in output, quindi **vedi** `Hello, Angela`.

☰ In sintesi

- `print()` all'interno della funzione fa **solo visualizzare** il risultato in output, senza permettere di usarlo altrove nel programma.
- `return` permetterebbe alla funzione di **restituire un valore** che puoi utilizzare successivamente.

Ritorno di più valori

Python ci consente di ritornare in output più valori contemporaneamente da una funzione.

La forma più comune per farlo è tramite una **tupla**.

Una funzione che restituisce una tupla può restituire più valori separati, e questi possono essere assegnati a variabili separatamente.



Python

```
1  # define a function returning multiple values(returning a tuple)
2  def operations(a: int, b: int) -> tuple[int, int]:
3
4      sum_result:int = a + b
5      diff_result:int = a - b
6      # Returns a tuple with two values
7      return sum_result, diff_result
8
9  # Assigns values to two variables
10 sum_value, diff_value = operations(10, 5)
11
12 print("Sum:", sum_value) # Output: Sum: 15
13
14 print("Difference:", diff_value) # Output: Difference: 5
15
16 print(type(operations(10,5)))
17
18 >>> <class 'tuple'>
```

Quando una funzione deve restituire più di un valore, Python offre diverse opzioni per farlo.

In Python, una funzione può restituire più valori utilizzando **tuple**, **liste** o **dizionari**.

1. Tupla:

una struttura dati che può contenere più valori, ed è immutabile, cioè

non puoi modificarne gli elementi una volta che la tupla è stata creata.

2. **Lista:**

una struttura dati che può anch'essa contenere più valori, ma è mutabile, quindi puoi aggiungere, rimuovere o modificare gli elementi.

3. **Dizionari:**

Permettono di restituire valori con chiavi descrittive, migliorando la leggibilità del codice.

Come detto poco sopra, in Python, se una funzione deve restituire più di un valore, solitamente si usa una **tupla**.

La sintassi per farlo è molto semplice, infatti puoi restituire i valori separandoli con delle virgole, e Python li restituirà automaticamente come una tupla.

La tupla di un elemento è l'elemento stesso:

Quando una funzione restituisce solo un singolo valore separato da una virgola, Python lo interpreta come una tupla di un solo elemento.

Questo accade perché, in Python, le tuple sono definite da una **virgola**, non dalla parentesi tonda. Pertanto, se restituisci solo un valore seguito da una virgola, Python lo considera come una tupla contenente quel valore.

≡ Esempio ▾



Python

```
1 def single_value():
2     return 5, # Questo è un ritorno di una tupla con un solo
               elemento
3
```

Anche se c'è un solo elemento, Python considererà `5,` come una tupla con un solo valore: `(5,)`.

Se omettessi la virgola, Python restituirà un singolo valore senza creare una tupla.

≡ Esempio ▾



Python

```
1 def single_value():
2     return 5
```

```

3
4 value = single_value()
5 print(type(value))
6
7 #Output:
8 <class 'int'>

```

Se invece restituisci una lista, Python ti permette di restituire più valori in una struttura mutabile, il che significa che i valori al suo interno possono essere modificati, aggiunti o rimossi.

☰ Esempio ▾



Python

```

1 def return_list(a, b):
2     return [a, b] # Restituisce una lista contenente i due
   valori
3

```

Stampa una tupla perché gli sto ritornando delle quantità definite secondo una sequenza ordinata e siccome questi valori non possono essere modificati perché è la funzione li può modificare, e li ritorna in modo diretto, Python li calcola come se fosse una tupla perché le tuple non sono modificabili.

Passare una Lista

In Python, una funzione può restituire più di un valore utilizzando tuple, liste o dizionari. Il metodo più comune è la restituzione di una in cui i valori sono separati da una virgola dopo l'istruzione `return`.

Come detto poco prima, spesso è utile passare una lista a una funzione, sia che contenga nomi, numeri o oggetti più complessi, come ad esempio i dizionari. Quando si passa una lista a una funzione, quest'ultima ha accesso diretto ai suoi contenuti.



Python

```

1 # define a function returning a list
2 def get_coordinates(x:int, y:int) -> list[float]:

```

```

3     return [x, y]
4
5     coords = get_coordinates(12.5, 45.8)
6
7     print(coords[0], coords[1], sep=", ")
8     >>> 12.5, 45.8
9
10    print(type(coords))
11    >>> <class 'list'>

```

Questa funzione `get_coordinates` prende in input due valori numerici `x` e `y` (di tipo `float`) e restituisce una **lista** (`return [x, y]`) contenente questi due valori. La notazione `-> list[float]` indica che il valore di ritorno è una lista di numeri in virgola mobile.

Con la variabile `coords = get_coordinates(12.5, 45.8)`, chiamiamo la funzione passandole `12.5` e `45.8`. La funzione restituisce la lista `[12.5, 45.8]`, che viene assegnata alla variabile `coords`.

Con `print(coords[0], coords[1], sep=", ")` stampiamo i due elementi della lista `coords`, separandoli con una virgola.

Poiché `coords` è una lista con due valori (`[12.5, 45.8]`), il primo elemento (`coords[0]`) è `12.5` e il secondo (`coords[1]`) è `45.8`.

Con `print(type(coords))` verifichiamo il tipo di `coords`. L'output `<class 'list'>` conferma che `coords` è una lista.

Modificare una lista in una funzione

Quando si passa una lista in una funzione, essa può modificare il contenuto della lista. Ogni cambiamento fatto alla lista che si trova dentro al corpo della funzione è permanente, e permette di lavorare in modo efficiente anche quando si ha a che fare con grandi quantità di dati.



Python

```

1  unprinted_designs:list[str] = ['phone case', 'robot pendant',
    'Obs']
2
3  completed_models:list[str]= []
4
5
6  while unprinted_designs:

```

```

7  current_designs = unprinted_designs.pop()
8  print(f"Printing Model:{current_designs}")
9  completed_models.append(current_designs)
10
11
12  print("\nThe following models have been print:")
13  for completed_model in completed_models:
14      print(completed_model)

```

In questo esempio

1. ho creato 2 liste:

- La prima lista `unprinted_designs` contiene tre stringhe: `'phone case'`, `'robot pendant'` e `'Obs'`.
- La seconda lista è una lista vuota `completed_model`, che serve per contare i modelli stampati.

2. Dopodiché uso il ciclo `while` per spostare gli elementi:

- `while unprinted_designs:`
Il ciclo `while` continua a eseguire il blocco di codice finché la lista `unprinted_designs` non è vuota.

👁 Info

In Python, una lista vuota è considerata "falsy", quindi il ciclo termina quando tutti gli elementi vengono rimossi.

3. Uso la funzione `.pop()`:

```
current_designs = unprinted_designs.pop()
```

- La funzione `.pop()` rimuove gli elementi dalla lista e li restituisce
 - Se `.pop()` viene chiamato senza argomenti, rimuove l'ultimo elemento della lista.
 - Il valore rimosso viene salvato nella variabile `current_designs`.
 - l'ordine di rimozione sarà: `Obs`, poi `robot pendant` e infine `phone case`

4. Stampo l'elemento in fase di elaborazione:

```
print(f"Printing Model:{current_designs}")
```

5. Sposto il modello stampato nella lista:

```
completed_models.append(current_designs)
```

- L'elemento rimosso da `unprinted_designs` viene aggiunto alla lista `completed_models`.

6. Dopo il `while` stampo i modelli completati:

```
print("\nThe following models have been print:")  
for completed_model in completed_models:  
    print(completed_model)
```

- Dopo che il ciclo `while` termina (cioè dopo che `unprinted_designs` è vuota), stampiamo tutti i modelli completati.

🔗 Ordine di esecuzione del codice >

1. Prima iterazione (`while unprinted_designs` non è vuota)

- `.pop()` rimuove `'Obs'`
- Stampa: `"Printing Model: Obs"`
- Aggiunge `'Obs'` a `completed_models`

2. Seconda iterazione

- `.pop()` rimuove `'robot pendant'`
- Stampa: `"Printing Model: robot pendant"`
- Aggiunge `'robot pendant'` a `completed_models`

3. Terza iterazione

- `.pop()` rimuove `'phone case'`
- Stampa: `"Printing Model: phone case"`
- Aggiunge `'phone case'` a `completed_models`

4. La lista `unprinted_designs` è ora vuota → il ciclo `while` termina.

5. Stampa finale della lista `completed_models`

Possiamo riorganizzare questo codice scrivendo due funzioni, ognuna delle quali svolge un compito specifico. La maggior parte del codice non cambierà; lo struttureremo solo in modo più ordinato.

La prima funzione si occuperà della stampa dei modelli, mentre la seconda riassumerà i modelli che sono stati stampati.



Python

```
1 def print_models(unprinted_designs: list[str], completed_models:  
    list[str]) -> None:
```

```

2
3     """Sposta i modelli non stampati alla lista dei modelli
completati."""
4
5     while unprinted_designs:
6
7         current_design = unprinted_designs.pop()
8
9         print("Printing model:", current_design)
10
11         completed_models.append(current_design)
12
13
14
15 def show_completed_models(completed_models: list[str]) -> None:
16
17     """Mostra tutti i modelli stampati."""
18
19     print("\nThe following models have been printed:")
20
21     for completed_model in completed_models:
22
23         print(completed_model)
24
25
26
27 # Liste iniziali
28
29 unprinted_designs: list[str] = ['phone case', 'robot pendant',
'Obs']
30
31 completed_models: list[str] = [] # Nome corretto
32
33
34
35 # Chiamata alle funzioni
36
37 print_models(unprinted_designs, completed_models)
38
39 show_completed_models(completed_models)

```

In questo caso:

1. Abbiamo definito la funzione `print_models()` con due parametri:

- una lista di designs che devono essere stampati (`unprinted_designs`)
 - una lista dei modelli completi (`completed_models`).
- Date queste due liste, la funzione simula il printing di ogni design andando a svuotare la lista `unprinted_design` e riempiendo la lista `completed_models` .

2. Definiamo la funzione `show_completed_models()` che ha un solo parametro:

- La lista `completed_models`

Definendo la lista, `show_completed_models()` va visualizzare il nome di ogni modello che viene stampato.

Come possiamo notare questo programma ha lo stesso output della [versione senza le funzioni](#), ma qui è più organizzato.

Il codice che fa buona parte del lavoro è stato suddiviso in due funzioni separate, il che lo rende più facile da comprendere

Passare un dizionario

In Python, una funzione può restituire più di un valore utilizzando tuple, liste o dizionari. Il metodo più comune è la restituzione di una

in cui i valori sono separati da una virgola dopo l'istruzione `return`.

Come detto sopra, le funzioni in Python possono restituire più valori utilizzando tuple, liste e dizionari.

Ciò è utile quando devo maneggiare strutture dati complicate.



Python

```
1  #define a function returning a dictionary
2  def get_user(myname:str, myrole:str) -> dict[str, str]:
3      return {"name": myname, "role": myrole}
4
5  user = get_user("Alice", "Admin")
6
7  print(user["name"]) # Output: Alice
8
9  print(user["role"]) # Output: Admin
10
11 print(type(user))
12 >>> Alice
13 >>> Admin
14 >>> <class 'dict'>
```

In questo caso:

1.

- **Definizione della funzione** `get_user()`
 - Prende due parametri: `myname` e `myrole`, entrambi stringhe.
 - Restituisce un dizionario con due chiavi: `"name"` e `"role"`, i cui valori sono i parametri passati alla funzione.
- **Assegnazione della funzione a una variabile**
 - `user = get_user("Alice", "Admin")`
 - La funzione viene eseguita e il risultato (un dizionario) viene memorizzato nella variabile `user`.
- **Accesso ai valori del dizionario**
 - `print(user["name"])` → Stampa `"Alice"`, il valore associato alla chiave `"name"`.
 - `print(user["role"])` → Stampa `"Admin"`, il valore associato alla chiave `"role"`.
- **Verifica del tipo di dato**
 - `print(type(user))` mostra `<class 'dict'>`, confermando che `user` è un dizionario.

Differenza tra argomenti e parametri ✓

Nell'esempio precedente, abbiamo definito la funzione `get_user()`, che accetta due parametri in input: `myname` e `myrole`.

Quando chiamiamo la funzione e le forniamo due valori (`"Alice"` e `"Admin"`), la funzione restituisce un dizionario con la formattazione corretta.

- **I parametri** `myname` e `myrole` sono **variabili definite nella funzione**, che rappresentano le informazioni di cui la funzione ha bisogno per svolgere il suo compito.
- **Gli argomenti** `"Alice"` e `"Admin"` sono **i valori effettivi passati alla funzione** al momento della chiamata.

In altre parole, quando chiamiamo `get_user("Alice", "Admin")`,

- L'argomento `"Alice"` viene assegnato al parametro `myname`.
- L'argomento `"Admin"` viene assegnato al parametro `myrole`.

A volte le persone usano i termini **argomenti** e **parametri** in modo intercambiabile. Non sorprenderti se vedi le variabili nella definizione di una funzione chiamate **argomenti**, o le variabili in una chiamata di funzione chiamate **parametri**.

Spiegazione >

Anche se i termini **argomento** e **parametro** hanno significati distinti in programmazione, capita spesso che vengano confusi o usati in modo errato.

- **Parametro:**
È la variabile dichiarata nella definizione della funzione. È un segnaposto per il valore che verrà passato alla funzione.
- **Argomento:**
È il valore effettivo che viene fornito quando si chiama la funzione.

Funzioni e parametri

Poiché una definizione di funzione può avere più parametri, una chiamata di una funzione potrebbe aver bisogno di argomenti multipli.

Esistono tre modi per passare argomenti a una funzione, assegnandoli ai rispettivi parametri

- **passare l'argomento per posizione**
- **passare l'argomento per keyword**
- **passare l'argomento per valori di default**

Quando gli argomenti vengono passati per posizione, gli argomenti nell'istruzione chiamante vengono abbinati ai parametri nell'intestazione della funzione

in base al loro ordine.

Ovvero:

- **il primo argomento viene passato al primo parametro.**
- **il secondo argomento viene passato al secondo parametro e così via.**

Argomento passato per posizione

Quando chiami una funzione, Python deve abbinare **ogni argomento della chiamata della funzione a un parametro nella definizione della funzione**.

Il modo più semplice per farlo è basato sull'ordine degli argomenti forniti.

I valori che vengono matchati in questo modo sono chiamati argomenti posizionali (o passato per posizione).



Python

```
1 def describe_person(name:str, age:int, city:str):
2     print(f"my name is{name}, I'm {age} years old and I live in
    {city}.")
3
4     # Chiamata alla funzione con passaggio per posizione
5     describe_person("Alice", 25, "Rome")
6     >>>
7     >>>"my name is Alice, I'm 25 years old and I live in Rome"
```

In questo esempio:

Ho tre parametri (name , age , city), che sono obbligatori e vengono passati **per posizione**.

- Il primo argomento passato è "Alice" , che corrisponde al parametro name (di tipo stringa).
- Il secondo argomento è 25 , che corrisponde al parametro age (di tipo intero).
- Il terzo argomento è "Rome" , che corrisponde al parametro city (di tipo stringa).

Se richiama la funzione ed invertissi i valori passati come argomenti nella chiamata della funzione, i valori verrebbero assegnati ai parametri in base alla loro posizione.



Python

```
1 def describe_person(name:str, age:int, city:str):
2     print(f"my name is{name}, I'm {age} years old and I live in
    {city}.")
3
4     # Chiamata alla funzione con passaggio per posizione
5     describe_person("Rome", 25, "Alice" )
6     >>>
7     >>>"my name is Rome, I'm 25 years old and I live in Alice"
```

Questo perché negli argomenti passati per posizione, l'ordine degli argomenti deve seguire lo stesso ordine dei parametri che sono stati scritti.

Difatti



Python

```
1 def greet(name:str, age:int) -> None:
2     print("Hi, my name is" + name + " and I'm " + str(age) + " years
    old!")
3     greet("Angela", 13)
```

in questo caso è giusto, ma se si invertisse l'ordine degli argomenti; ad esempio scrivo prima l'intero 13 e poi la stringa "Angela" mi darebbe errore perché non posso concatenare un intero con delle stringhe in Python.

Argomento passato per keyword

Un argomento passato per keyword è una coppia nome-valore che viene passata a una funzione.

Associando direttamente il nome al valore all'interno dell'argomento, si evita ogni possibile ambiguità.

Questa tipologia è utile perché non c'è bisogno di ordinare correttamente gli argomenti nella chiamata della funzione e chiariscono il ruolo di ciascun valore all'interno della chiamata stessa.

Per fare un esempio concreto, riprendiamo la funzione `describe_person()`:



Python

```
1 def describe_person(name:str, age:int, city:str) ->Any:
2
3     greetings = f"Hello my name is {name}, I'm {age} years old
    and I live          in {city}"
4     return greetings
5
6     print(describe_person(age=25, city="Rome", name="Carlo"))
```

Posso invocare la funzione passando gli argomenti per keyword, ad esempio `age=25`, `city="Rome"` e `name="Carlo"`.

Quando uso solo argomenti per keyword, **l'ordine non è importante:**

perché ogni valore viene associato direttamente al parametro corrispondente in base al nome.

Anche se nella chiamata della funzione gli argomenti vengono passati in un ordine diverso rispetto alla definizione, Python li abbina correttamente ai loro parametri.

Posso anche mischiare argomenti **posizionali e per keyword**, ma in questo caso **gli argomenti posizionali devono sempre venire prima di quelli per keyword**.

Non posso mettere un argomento posizionale dopo un **keyword argument**, perché Python non saprebbe più come abbinarli.

Per fare un ulteriore esempio:



Python

```
1  def describe_person(name: str, age: int, city: str) -> str:
2      greetings = f"Hello, my name is {name}, I'm {age} years old
   and I live in {city}."
3      return greetings
4
5
6  # Solo keyword
7  print(describe_person(age=25, city="Rome", name="Carlo"))
8
9  #Posizionale + keyword
10 print(describe_person("Alice", age=30, city="Milan"))
11
```

Argomenti passati per valori di default

Quando si scrive una funzione, è possibile definire un valore di default per ogni parametro.

Se un argomento per un parametro viene fornito nella chiamata della funzione, Python utilizza il valore di quell'argomento.

Se l'argomento non viene fornito, Python usa il valore di default definito per quel parametro.

Pertanto, quando si definisce un valore di default per un parametro, è possibile escludere l'argomento corrispondente dalla chiamata della funzione.

L'uso di valori di default semplifica la funzione e può aiutare a chiarire l'utilizzo tipico di essa.

La sintassi è:



Python

```
1 def function(par1, par2, par3=value3, par4=value4):
```

Prendendo come esempio questo blocco di codice, possiamo dire che:

- I primi due parametri sono obbligatori perché non hanno i valori di default. Quindi quando si chiama la funzione Python si aspetta che gli si fornisca un argomento per ogni parametro obbligatorio (cioè per `par1` , `par2`).
- Gli ultimi due parametri (`par3` e `par4`) hanno due valori di default. Questo significa che puoi scegliere di fornire valori per questi parametri nella chiamata della funzione, ma **se non lo fai**, Python userà i valori di default definiti nella funzione.

Perché i primi due parametri sono obbligatori >

1. **Posizione e ordine:** Python abbina gli argomenti alle variabili della funzione in base alla loro **posizione**. Gli argomenti devono essere passati nell'ordine in cui sono definiti nella funzione. Per esempio, se chiami la funzione come `function(1, 2)` , il primo argomento (1) viene assegnato a `par1` e il secondo (2) a `par2` . Se non fornisci questi argomenti, Python non sa cosa fare con `par1` e `par2` , quindi si verifica un errore.
2. **Impossibilità di passare solo gli argomenti opzionali:** In Python, non puoi passare solo i parametri opzionali (quelli con valori di default) senza passare prima i parametri obbligatori. Quindi, devi sempre passare prima i parametri senza valore di default (posizionali) e poi puoi aggiungere quelli opzionali se lo desideri.

Ad esempio:



Python

```
1 def total(w, x, y=10, z=20):
2     return (w ** x) + y + z
3
4 # calling function total, while omitting the last two values
5 total(2, 3)
6 # Output: 38 -> calculated as 2^3 + 10 + 20
7
8 # calling function total, while omitting the last values
9 total(2, 3, 4)
10 # Output: 32 -> calculated as 2^3 + 4 + 20
```

```

11
12  # calling function total with 4 input values
13  total(2, 3, 4, 5)
14  # Output: 17 -> calculated as 2^3 + 4 + 5

```

In questo blocco di codice i primi due parametri (`w` e `x`) sono obbligatori, mentre i successivi due parametri (`y = 10` e `z = 20`) hanno dei valori di default.

- Con il `return` mi ritorna in output la potenza tra i primi due parametri e l'addizione, del risultato della potenza tra `w` e `x` , con gli ultimi due parametri.

Prima chiamata

- invoco la funzione `total()` ed assegno dei valori passati come argomenti posizionali a primi due parametri:

`w = 2` e `x = 3`

$2^3 + 10 + 20 = 38$

Seconda chiamata:

- chiamo la funzione `total()` ed assegno sempre ai primi due parametri due valori passati come argomenti posizionali:

`w = 2` e `x = 3`

- Nella chiamata cambio il valore della variabile `y` da `10` a `4` :

`y = 4`

$2^3 + 4 + 20 = 32$

Terza chiamata:

- Nella chiamata cambio il valore della variabile `z` da `20` a `5` :

`w = 2` , `x = 3` , `y = 4` , `z = 5`

$2^3 + 4 + 5 = 17$.

Ordine dei Parametri e degli Argomenti in una Funzione

Nella definizione di una funzione in Python, i parametri possono essere **obbligatori**(posizionali e keyword) o **opzionali**(valori di default).

Tra i parametri obbligatori troviamo quelli **posizionali** (che devono essere passati in base all'ordine) e quelli **keyword-only** (che devono essere passati con il nome esplicitato). I parametri opzionali, invece, hanno un valore di **default** e vengono usati solo se non viene fornito un argomento esplicito.

Ordine corretto dei parametri nella definizione di una funzione

1. Parametri posizionali obbligatori

2. Parametri opzionali con valore di default
3. `*args` (per un numero variabile di argomenti posizionali)
4. Parametri keyword-only obbligatori (definiti dopo `*`)
5. Parametri keyword con valore di default
6. `[[#Funzioni e `kwargs` | `kwargs`]]` (per un numero variabile di argomenti passati per keyword)

Quindi:

- Gli **argomenti posizionali** devono essere forniti per primi, rispettando l'ordine dei parametri.
- Gli **argomenti keyword** possono essere usati dopo quelli posizionali, specificando esplicitamente il nome del parametro.

☰ Esempio di una funzione che combina parametri posizionali, keyword e di default >



Python

```
1 def descrivi_animale(nome, specie='cane'):
2     print(f"\nIl mio animale è un {specie}.")
3     print(f"Si chiama {nome.title()}")
4     # Chiamate equivalenti della funzione:
5     descrivi_animale('willie')      # Output: cane di nome Willie
6     descrivi_animale(nome='willie') # Stesso output
7     descrivi_animale('harry', 'criceto')
8     descrivi_animale(nome='harry', specie='criceto') # Stesso
9     descrivi_animale(specie='criceto', nome='harry') # Stesso
        output
```

Nota

Le chiamate funzionano perché gli argomenti vengono passati rispettando la posizione o usando il nome del parametro.

Funzioni senza parametri

In Python, una funzione può essere definita senza parametri se non ha bisogno di ricevere dati in ingresso per eseguire un'azione.

Queste funzioni sono utili quando devono sempre svolgere un'operazione fissa e indipendente dagli input esterni.



Python

```
1 def hello():
2     print('Hello')
3 hello()
4 >>> Hello
```

Quindi alcune funzioni possono essere definite ed eseguite senza ricevere alcun valore di ingresso o parametro.

In Python, le funzioni non sempre richiedono argomenti per eseguire un'azione. Questi tipi di funzioni sono utili quando un'attività deve essere eseguito in modo standard, senza dipendere da input esterni.

Questo può essere utile quando devo lavorare con variabili globali o valori costanti.

≡ Esempio con un valore restituito



Python

```
1 def get_pi():
2     return 3.14159
3
4 pi_value:float = get_pi()
5 print("The value of pi is:", pi_value)
6 >>> The value of pi is:3.14159
```

In questo caso la funzione `get_pi()` non ha bisogno di parametri perché restituisce sempre lo stesso valore costante

Chiamare ed eseguire le funzioni

Per usare una funzione, in python, è necessario **chiamarla o invocarla**.

Quando un programma chiama una funzione:

1. il controllo del programma viene trasferito alla funzione chiamata.

2. La funzione viene eseguita fino alla fine o fino all'incontro di un'istruzione `return` .
3. Il controllo torna al chiamante. cioè alla riga da cui la funzione è stata chiamata.

Come viene eseguito questo programma?

L'interprete legge lo script nel file, riga per riga, a partire dalla riga 1.

Quando legge l'intestazione della funzione alla riga 1, memorizza la funzione con il suo corpo (righe 1-riga n) in memoria.

- **Ricordate che la definizione di una funzione definisce la funzione, ma non ne determina l'esecuzione.**



Python

```
1  def max(num1:int, num2:int)->int:
2      if num1>num2:
3          return num1
4      else:
5          return num2
6  #chiama la funzione max
7  numMax:int = max(3,4)
8  print(numMax)
9
10  >>>4
```

Come viene eseguito il codice?

1. L'interprete legge il file Python riga per riga, partendo dalla riga 1.
2. Alla riga `def max(num1: int, num2: int)` , **Python definisce la funzione**, ma non la esegue.
3. La funzione `max(3,4)` viene chiamata alla riga `numMax = max(3, 4)` .
4. Il valore `3` viene passato a `num1` e il valore `4` a `num2` .
5. Il controllo passa alla funzione `max()` , che confronta i due numeri e restituisce `4` .
6. Il valore restituito viene assegnato alla variabile `numMax` .
7. Infine, `print(numMax)` stampa `4` a schermo.

👁 **Concetti chiave** ✓

- Prima della chiamata `max(3,4)`, il controllo si trova nel programma principale.
- Durante l'esecuzione della funzione, il controllo passa alla funzione `max()`.
- Quando la funzione termina, il controllo torna all'istruzione chiamante (`numMax = max(3,4)`).

Call Stacks(Pila di chiamate)

Ogni volta che una funzione è invocata, **il sistema crea un record di attivazione che memorizza gli argomenti e le variabili per la funzione e colloca il record di attivazione in un'area di memoria nota come Call Stack(stack di chiamate).**

Quando una funzione chiama un'altra funzione:

- Il record di attivazione del chiamante viene mantenuto intatto
 - Viene creato un nuovo record di attivazione per la nuova chiamata di funzione.
- Quando una funzione termina il suo lavoro e restituisce il controllo al chiamante, il suo record di attivazione viene rimosso dalla pila (stack) delle chiamate.

Struttura e comportamento del Call Stack

Il Call Stack segue una strategia **Last-In, First-Out (LIFO)**:
ovvero l'ultimo record inserito è il primo a essere rimosso.

Esempio: Supponiamo che la funzione `m1` chiami la funzione `m2`, e successivamente `m3`:



Python

```
1  def m3:
2      print("Esecuzione di m3")
3
4  def m2():
5      print("Esecuzione di m2")
6
7  def m1():
8      print("Esecuzione di m1")
9      m2
10
```

```
11 m1()
```

Passaggi dell'esecuzione:

1. Il sistema runtime inserisce nello stack il record di attivazione di `m1`.
2. Poi inserisce quello di `m2`.
3. Infine, inserisce quello di `m3`.

L'ordine di rimozione sarà:

- Una volta che `m3` ha terminato la sua esecuzione, il suo record di attivazione viene rimosso dallo stack.
- Dopo che `m2` ha terminato, il suo record viene rimosso.
- Infine, quando `m1` termina, anche il suo record viene rimosso.

Parametri e Call Stack

Per quanto riguarda i parametri passati alle funzioni:

- Il primo parametro inserito nel record di attivazione di una funzione è anche il primo a essere utilizzato dalla funzione stessa.
- Tuttavia, i record di attivazione delle funzioni seguono il principio LIFO nel Call Stack.

Questa struttura è fondamentale per la gestione delle chiamate di funzione e consente il corretto flusso di esecuzione dei programmi.

Esempi di Call Stack >

```
Python
1 def m3():
2     print("Esecuzione di m3")
3
4 def m2():
5     print("Esecuzione di m2")
6     m3()
7
8 def m1():
9     print("Esecuzione di m1")
10    m2()
11
12 m1()
```

```
13 >>>"Esecuzione di m1
14 >>>Esecuzione di m2
15 >>>Esecuzione di m3
16
```

Passaggi dell'esecuzione:

1. `m1()` viene chiamata e il suo record di attivazione viene aggiunto allo stack.
2. `m1` chiama `m2()`, quindi il record di `m2` viene aggiunto sopra `m1`.
3. `m2` chiama `m3()`, quindi il record di `m3` viene aggiunto sopra `m2`.
4. `m3` termina, quindi il suo record viene rimosso.
5. `m2` termina, quindi il suo record viene rimosso.
6. `m1` termina, quindi il suo record viene rimosso.

Esempio 2: Passaggi dell'esecuzione:



Python

```
1 def somma(a, b):
2     return a + b
3
4 def moltiplica(x, y):
5     return somma(x, y) * 2
6 risultato = moltiplica(3, 4)
7 print(risultato)
8
```

Spiegazione

- `moltiplica(3,4)` viene chiamata e il suo record di attivazione viene aggiunto allo stack.
- `moltiplica` chiama `somma(3,4)`, quindi il record di `somma` viene aggiunto a sopra `moltiplica`.
- `somma` termina e restituisce 7, quindi il suo record viene rimosso
- `moltiplica` calcola `7*2` e restituisce 14, quindi il suo record viene rimosso.
- valore `14` viene stampato a schermo.

Questa struttura è fondamentale per la gestione delle chiamate di funzione e consente il corretto flusso di esecuzione dei programmi.

Funzioni e Argomenti variabili

In alcuni casi, non sappiamo esattamente quanti argomenti una funzione dovrà accettare. Fortunatamente, Python permette di raccogliere un numero arbitrario di argomenti da una chiamata di funzione, utilizzando `*args` per gli argomenti posizionali e `**kwargs` per quelli con nome.

Funzioni e `*args`

In Python io posso creare delle funzioni che accettano variabile di argomenti. Ad esempio immaginiamo di voler creare una funzione che somma due numeri:



Python

```
1 def add(a, b):  
2     return a + b  
3  
4 print(add(2, 3))  
5 print(add(2, 3, 4))
```

il primo print funziona poiché ho definito dei valori posizionali nella chiamata della funzione che ho messa come argomento del `print()`, mentre

`print(add(2,3,4))`, **mi torna errore perché l'ultimo argomento è mancante nella definizione della funzione** `add`.

Quindi se voglio aggiungere altri argomenti devo scrivere `*args`:



Python

```
1 def add(a, b):  
2     return a + b  
3 print(add(2, 3))  
4  
5 add(*args 1,2,4,5,10,56, "Hello")
```

Quando richiamo la funzione e le passo `*args`, posso fornirle un numero variabile di argomenti, compatibili con le operazioni svolte dalla funzione (fino al limite della memoria disponibile)

Per dirla in modo tecnico:

`*args` è una **collezione arbitraria di argomenti posizionali**:
cioè permette a una funzione di accettare un numero variabile di argomenti
senza specificare quanti in anticipo.

✓ **Caratteristiche principali di `*args` :**

- **Accetta un numero arbitrario di argomenti**
- **Gli argomenti sono trattati come una tupla all'interno della funzione**
- **Sono argomenti posizionali:**
L'ordine con cui vengono passati è importante

✗ **Limiti di `*args`**

Non può gestire **argomenti con nome**(**Keyword arguments**).
In questo caso si deve usare il **`**kwargs`** che vedremo più avanti

L'operatore `*` >

`*args` è un modo generico che permette a una funzione di accettare un numero variabile di argomenti.

Quando si usa `*args` in una funzione, **Python raccoglie tutti gli argomenti passati alla funzione e li inserisce in una tupla, permettendoti di trattare un numero indefinito di argomenti.**

Ad esempio :



Python

```
1 myList:list[int] = [1,2,3,4,5,6,7,8,9]
2 print(*myList)
```

Mi stampa gli elementi della lista senza le parentesi quadre, questo perché quando si mette l'operatore `*` davanti a una lista (in questo caso `myList`, ma funziona anche con le tuple) spacchetta la lista.

Questo significa che invece di passare la lista come un singolo argomento alla funzione `print()`, gli elementi della lista vengono passati separatamente come argomenti distinti.

In sintesi, l'operatore `*` spacchetta la lista e la "espande" in singoli argomenti

per la funzione `print()` , che li gestisce e li stampa separati da uno spazio per default.

Ma cosa succede se l'operatore `*` viene posto prima di un parametro nella definizione di una funzione in Python?

Prendiamo ad esempio questo blocco di codice:



Python

```
1 def make_pizza(size, *toppings):
2     """Summarize the pizza we are about to make."""
3     print(f"\nMaking a {size}-inch pizza with the following
4         toppings:")
5     for topping in toppings:
6         print(f"- {topping}")
7
8 # Chiamate della funzione
9 make_pizza(16, 'pepperoni')
10 make_pizza(12, 'mushrooms', 'green peppers', 'extra cheese')
11
12 #Output
13 >>> Making a 16-inch pizza with the following toppings:
14 >>> pepperoni
15 >>> Making a 12-inch pizza with the following toppings:
16 >>>- mushrooms
17 >>>- green peppers
18 >>>- extra cheese
```

Spiegazione:

- `size` è un argomento posizionale (la dimensione della pizza).
- `*toppings` raccoglie tutti gli argomenti successivi come una tupla, che rappresentano i condimenti (toppings) della pizza.
- Le chiamate a `make_pizza` mostrano due esempi:
- La prima crea una pizza da 16 pollici con il topping `'pepperoni'`.
- La seconda crea una pizza da 12 pollici con tre topping: `'mushrooms'`, `'green peppers'` e `'extra cheese'`.

Quindi l'operatore `*` permette alla funzione di accettare un numero arbitrario di topping per ogni pizza, come vediamo nel secondo esempio.

Come per `*args`, è utile passare un numero arbitrario di argomenti a una funzione. Tuttavia, non sempre si può sapere in anticipo che tipo di informazioni verranno passate.

In questi casi, si può scegliere di usare funzioni che accettano coppie chiave-valore fornite dall'istruzione che le chiama.

Un esempio di utilizzo è la creazione di profili utente:

si sa che si riceveranno informazioni su un utente, ma non si è sicuri di quale tipo di dati verranno forniti.

Qui entra in gioco il `**kwargs`:

è usato per raccogliere un numero arbitrario di argomenti passati keyword (key-value pairs) in una funzione.

✓ Differenza tra `*args`, `**kwargs`

1. `*args`:

restituisce una tupla di elementi, cioè raccoglie argomenti posizionali in una tupla.

2. `**kwargs`:

raccoglie argomenti passati come chiavi e valori in un dizionario.

Un esempio di utilizzo è la creazione di profili utente:

si sa che si riceveranno informazioni su un utente, ma non si è sicuri di quale tipo di dati verranno forniti..

Quindi viene trattato come un dizionario infatti posso usare i metodi di un dizionario.



Python

```
1  def total_price(**kwargs):
2      total:float = 0
3      for product, price in kwargs.items():
4          print(f"{product}: {price}€")
5          total += price
6      return round(total, 2)
7
8  print(total_price(coffee=2.99, cake=4.55, juice=2.99))
9  >>>
10 coffee: 2.99€
11 cake: 4.55€
```

```
12 juice: 2.99€
13 Total: 10.53€
```

Quindi è come se trattasi un dizionario.

L'operatore `**` >

L'operatore `**` in Python, quando usato nei parametri di una funzione, raccoglie gli argomenti passati per **chiave-valore** e li memorizza in un dizionario.

 `*args`, `*kwargs` sono nomi convenzionali

`*args` ed `**kwargs` non sono parole chiave riservate, ma solo nomi convenzionali.

Puoi sostituirli con qualsiasi altro nome significativo, purché rispetti la sintassi.

In realtà i nomi `args` ed `kwargs` sono nomi convenzionali ma non definiscono nulla:

sono gli operatori `*` ed `**` a definire il funzionamento degli argomenti variabili.

Esempi pratici dei due operatori >



Python

```
1 def somma_totale(*numeri):
2     """Somma tutti i numeri passati come argomenti
   posizionali."""
3     return sum(numeri)
4
5 print(somma_totale(1, 2, 3, 4)) # Output: 10
```

Qui `*numeri` fa la stessa cosa di `*args`.



Python

```
1 def dettagli_auto(modello, marca, **specifiche):
2     """Restituisce un dizionario con i dettagli dell'auto."""
3     specifiche["modello"] = modello
4     specifiche["marca"] = marca
```

```
5 return specifiche
6 auto = dettagli_auto("Golf", "Volkswagen", >>colore="blu",
7                       anno=2020)
8 print(auto)
```

Qui `**specifiche` raccoglie le informazioni extra come farebbe `**kwargs`.

In conclusione, puoi usare qualsiasi nome al posto di `args` e `kwargs`, l'importante è mantenere il `*` e ``` per il corretto funzionamento**. 

Built-in functions

come detto prima sono le funzioni di default date da python.

- `print()` : Visualizza dati sulla console.
- `len()` : Restituisce la lunghezza di un oggetto (es. stringa o lista).
- `max()` : Restituisce l'elemento più grande in un iterabile.
- `min()` : Restituisce l'elemento più piccolo in un iterabile.
- `sum()` : Restituisce la somma di tutti gli elementi in un iterabile.
- `abs()` : Restituisce il valore assoluto di un numero.
- `round()` : Arrotonda un numero a un numero specificato di cifre decimali.
- `sorted()` : Restituisce una nuova lista ordinata dagli elementi di un iterabile.
- `range()` : Genera una sequenza di numeri.